



Plain Text Files

Part 3: Working With CSV Files

Some files use comma separated values (or tab separated values) to store data in tables.

CSVs

A **CSV**, or **Comma-Separated Values** file, stores tabular data as plain text.

Each line usually represents one row, and commas separate the values into columns.

CSV files are commonly used for spreadsheets, exported data, grades, inventories, survey results, and data shared between different programs.

name,score,grade

Alex,82,B

Sam,91,A

Jordan,76,C

import csv

Python includes a built-in **csv** module for working with CSV files.

It provides tools for reading rows, writing rows, and working with headers.

The module must be imported before using these tools.

```
import csv
```

Opening a CSV File in **r** Mode

A CSV file can be opened in read mode using **"r"**.

The **with** statement automatically closes the file when the indented block finishes.

At this point, the file is open but has not yet been read as CSV data.

```
import csv

with open("students.csv", "r") as file:
    pass
```

Creating a Reader Object

The `csv.reader()` function creates a **reader object** from an open CSV file.

This reader separates each row into a list of values.

The reader can then be used to access the data one row at a time.

```
import csv

with open("students.csv", "r") as file:
    reader = csv.reader(file)
```

Iterating Over the Reader Object

A reader object can be used in a loop to access each row in the CSV file.

Each row is returned as a list.

This allows individual values to be accessed using normal list indices.

```
import csv

with open("students.csv", "r") as file:
    reader = csv.reader(file)

    for row in reader:
        print(row)
```

OUTPUT

```
['name', 'score', 'grade']
['Alex', '82', 'B']
['Sam', '91', 'A']
['Jordan', '76', 'C']
```

Using a DictReader

`csv.DictReader()` uses the first row of the CSV file as column names.

Each remaining row is returned as a dictionary instead of a list.

This allows values to be accessed using meaningful keys instead of numeric indices.

```
import csv

with open("students.csv", "r") as file:
    reader = csv.DictReader(file)

    for row in reader:
        print(row["name"], row["score"])
```

OUTPUT

```
Alex 82
Sam 91
Jordan 76
```

Writing CSV Files in **w** Mode

A CSV file can be opened in write mode using **"w"**.

The **newline=""** argument helps prevent extra blank lines from appearing between rows on some systems.

Write mode will replace the contents of an existing file with the same name.

```
import csv

with open("students.csv", "w", newline="") as file:
    pass
```

Creating a Writer Object

The `csv.writer()` function creates a writer object for an open CSV file.

The writer provides methods for adding rows to the file.

Each row is usually provided as a list.

```
import csv

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
```

Writing Single Rows

The `writerow()` method writes one row to the CSV file.

Each item in the list becomes a separate column.

It can be called multiple times to build the file one row at a time.

```
import csv

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)

    writer.writerow(["name", "score", "grade"])
    writer.writerow(["Alex", 82, "B"])
```

CSV Output

```
name,score,grade
Alex,82,B
```

Writing Multiple Rows with a List

The `writerows()` method writes several rows at once.

The data is usually stored as a list containing smaller lists.

Each inner list becomes one row in the CSV file.

```
import csv

rows = [
    ["name", "score", "grade"],
    ["Alex", 82, "B"],
    ["Sam", 91, "A"],
    ["Jordan", 76, "C"]
]

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(rows)
```

Writing from a Dictionary with Headers

`csv.DictWriter()` writes rows using dictionaries.

The `fieldnames` define the column headers, and `writeheader()` writes those headers to the file.

Each dictionary can then be written using `writerow()`.

```
import csv

with open("students.csv", "w", newline="") as file:
    headers = ["name", "score", "grade"]

    writer = csv.DictWriter(file, fieldnames=headers)

    writer.writeheader()
    writer.writerow({
        "name": "Alex",
        "score": 82,
        "grade": "B"
    })
```

CSV Output

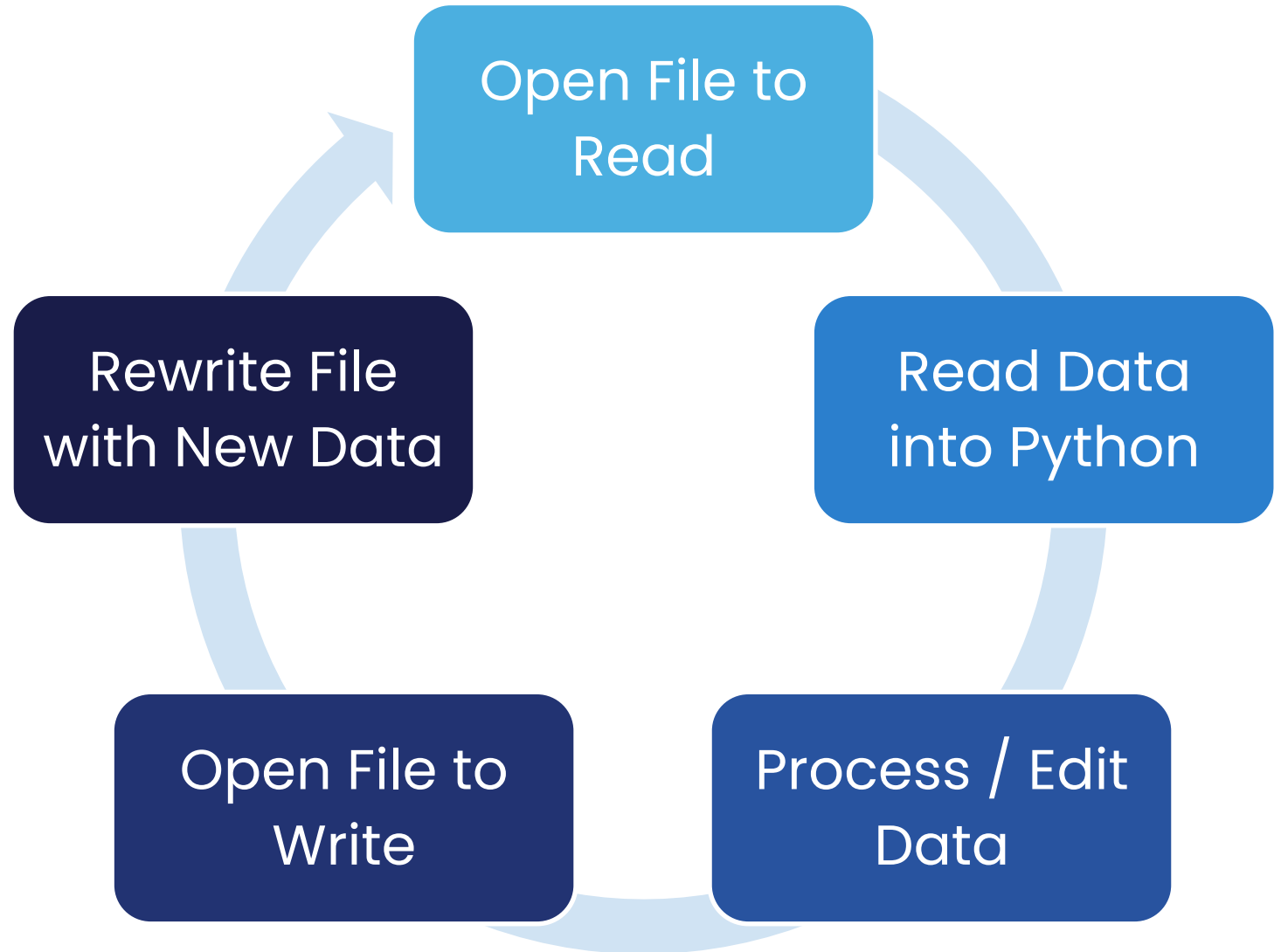
```
name,score,grade
Alex,82,B
```

Reading and Writing Files

A common file workflow is **read, process, write**.

The program first reads data from a file and stores it in Python.

It then processes or modifies that data before writing the updated information back to a file.



```
import csv

students = []

with open("students.csv", "r") as file:
    reader = csv.DictReader(file)

    for row in reader:
        row["score"] = int(row["score"]) + 5
        students.append(row)

with open("students.csv", "w", newline="") as file:
    headers = ["name", "score", "grade"]

    writer = csv.DictWriter(file, fieldnames=headers)

    writer.writeheader()
    writer.writerows(students)
```

Wrap Up

CSV files store tabular data as plain text, with each line representing a row and commas separating values into columns.

Python's csv module provides tools such as `reader()`, `DictReader()`, `writer()`, and `DictWriter()` for working with this data.

CSV files are commonly opened in `"r"` mode to read data and `"w"` mode with `newline=""` to write data.

A common workflow is to **read** the file, **process** the data in Python, and then **write** the updated data back to a file.